

Online Numerical Approximation for UH Leduc Poker

Mark Jaschke

Department of Aerospace
University of Colorado, Boulder
Boulder, United States
mark.jaschke@colorado.edu

Cristian Montelongo

Department of Aerospace
University of Colorado, Boulder
Boulder, United States
crmo5336@colorado.edu

Ikenna Olelewe

Department of Aerospace
University of Colorado, Boulder
Boulder, United States
lkol6839@colorado.edu

Abstract: This project compares the utility of a Partially Observable Monte Carlo Planning (POMCP) algorithm against a Hindsight Optimization (HOP) algorithm to solve a POMDP modeled on a simplified poker game. POMCP proved to be the more successful algorithm winning over 55% of hands for both our simplified (Leduc Poker) and more advanced POMDPs (UH-Leduc Poker).

Keywords—POMDP, POMCP, Leduc Poker, HOP, UH-Leduc Poker

I. INTRODUCTION

Poker is a strategic game which tests the confidence and mettle of players engaged in its execution. While simple in its appearance, it holds a degree of sophistication in which its participants must actively assess and evaluate the statistical prospects of their success during the game. Due to the complicated nature of the game in practice, poker can serve as an excellent use case for demonstrating the application of a Partially Observable Markov Decision Process (POMDP). This paper focuses on a simplified version of Texas Hold’Em poker which is known as Leduc poker. These poker variants both include community cards as well as private cards known only by the given player.

II. BACKGROUND RELATED WORK

The game of Poker is a comparison-based card game in which players attempt to obtain the best “hand”, or collection of cards, based on the games set hierarchy. In Texas Hold’Em, participants seek to obtain the best hand, as compared against fellow players. Each round consists of players taking one of the following three actions: A call or check in which a player matches the reward of the community pool, a raise in which a player increases the value of the initial bet in a round, and a fold in which a player elects to dismiss themselves from the game and forfeits the funds or reward they’ve added to the betting pool.

The uncertainty in Texas Hold’Em comes from the lack of knowledge that each player has of other participants hands. This element of uncertainty adds risk to the game as player confidence is determined by several factors, including their hand, the community cards, the suspected or perceived confidence of other players, and the reward accumulated by the multiple rounds of betting.

The goal of this project is to create a POMDP model to decide the best response to a given hand or a given environment in a Poker match. The goal is to have a model that works for 52 Card-Texas Hold-Em Poker, but the more achievable, intermediate goals, will be detailed further in the next sections.

The POMDP estimated reward for each hand or state of the environment will scale based on the number of chips that have been bet. Therefore, the reward environment evolves throughout the game. Once the reward environment is updated, our model then chooses the optimal response action based on the given environment.

Our baseline model goals are to achieve above a 50%-win rate while playing basic Leduc Poker (6 cards), against a singular static policy opponent. The more advanced project goal will use the slightly more complex game of UH (University of Houston) Leduc Poker (18 cards) and will be able to generate an optimal response to a given hand or environment. The stretch goal of Texas Hold’Em with a 52-card deck will be left for future work.

Leduc Poker is a simplified version of Poker. The game features six cards consisting of pairs of Kings, Queens, and Jacks. In Leduc Poker, only two players are featured as participants, the number of rounds is limited to two, and the number of betting rounds is limited to two. After the second round, the community card is revealed, followed by another round of betting. At the conclusion of the game a showdown occurs and the player with the same “rank” as the community card is declared the winner. If neither player possesses the same rank as the community card, then the player whose hand features the card with the higher rank is declared the winner.

The UH iteration of Leduc Poker features 18 cards instead of the original six, consisting of 4 Aces, 4 Kings, 6 Queens, and 4 Jacks. The larger card deck or state space and especially the inclusion of 2 additional Queens to the deck adds a layer of sophistication to the decision-making approaches that players may take during the game.

A. Related Published Works

The work of S. R. Parth [4], demonstrated how reinforcement learning, using the Q-Learning algorithm, could be implemented to learn poker, specifically UH Leduc poker. For our work, we decided to utilize the same, more advanced, version of Leduc poker, to demonstrate the robustness and capabilities of our algorithm. The work of Parth [4] was also a great introduction for the challenges involved in setting up a poker POMDP and the proper steps involved with building up and testing an algorithm to increasing levels of complexity.

For utilizing hindsight optimization, we referenced the work of Yoon et al. [6] to see how their hybrid POMDP

model functions, and the pitfalls of hindsight optimization within their modeling. We utilized this paper to compare how hindsight optimization interacted with our POMDP to determine if our model met similar pitfalls.

The work of Silver and Veness [5] was utilized when searching for an additional algorithm type to learn our poker model. Their paper described the efficiency and utility of the POMCP algorithm for large scale POMDPs like our poker formulation.

While there have not been specific papers detailing the utilization of Hindsight Optimization for poker specifically, Bowling et al. [1] approach poker with a similar, yet backwards approach, to applying the Counterfactual Regret Minimization (CFR) for poker. Instead of trying to choose the best action in hindsight, CFR analyzes the regret of NOT choosing the best action in hindsight. This helped view the application of HOP in Leduc Poker through a different lens and understanding what caused the under-performance of HOP.

The work of Gilpin and Sandholm [2], delves into the intricacies of game theory for “Rhode Island Hold’Em” a simplified variant of Texas Hold’Em. This paper was used to determine if we wanted to attempt a game theory implementation in the project as compared to focusing on creating a more elaborate POMDP for Leduc and UH Leduc Poker. Given the focus areas in this semesters DMU course it was decided that the various POMDP definitions were the better approach for our project, but this source can be used to direct further work in this field of research.

III. PROBLEM FORMULATION

We begin problem formulation for the most basic of our stratified goals, Leduc Poker. As previously stated, our goal here is to establish a functioning baseline model for which advanced iterations of the Poker simulator can be derived from. We use the programming language Julia as the language of choice for creating this model.

A. Implementation of Leduc Hold’Em

The stochastic implementation of this game followed a deterministic model we had developed to test some basic functionalities and observe whether our baseline model had the correct features to be demonstrated. Unlike a deterministic version of Leduc Poker, the stochastic version is suitable for a POMDP problem formulation as it incorporates uncertainty through player actions following a probability distribution.

B. Policy Dictionary

Each player in the stochastic model of this game has the following options: Fold, Call, and Raise. The player choices are mapped to a series of probabilities corresponding to the cards they are dealt at the beginning of the game. A mapping of the probabilities is featured below:

Card Drawn	p_fold	p_call	p_raise
Jack	1/3	1/3	1/3

Queen	.20	.50	.30
King	.10	.50	.40

Table 1: Table of Probabilities

Player behavior can be modified by changing probabilities in the policy.

C. Action Space

During every betting round of the game, the acting player’s card is drawn based off of a uniform random number “r” from the range [0, 1] and maps it to an action using cumulative probability thresholds:

Threshold	Action
If $r < p_fold$	Fold
If $r < p_fold + p_call$	Call
Otherwise	Raise

Table 2: Leduc Simulator Player Actions

D. Edge Case Resolution

We have also incorporated solutions to our model to address extreme edge cases where player actions occur outside of expected norms. The first edge case is folding as the opening action of the betting round. This is an irrational decision as a player would never fold for free. To address this problem, we introduce the action of check in the game. This check action is defined outside of the dictionary so as to not interfere with the stochastic model we defined governing all player actions. Another solution we introduce is to limit the raise action to only betting \$1 in the betting round and treating actions independent of the previous round of betting, meaning that the value from the previous bet will not carry on to the current round of betting. We do this to remain consistent with the rules established in Leduc Poker.

As we previously mentioned, all raises are limited to the value of \$1 over the current last bet. If the player commits to the action of raise, but cannot afford the raise amount, they must defer to calling instead. If the player cannot afford to call and match the existing bet, they must elect to fold. This process is included to ensure that the simulation does not fail in its execution due to insufficient funds in the game.

Once a betting round commences, any of the following actions will lead to the conclusion of the round:

1. A player folds
2. A player calls a bet or raise
3. Both players check in sequence
4. The action counter reaches 4

The final reason for round termination is to prevent infinite loops from occurring.

A coin flip at the beginning of the game determines which player acts first. The same player remains first in both betting rounds to maintain consistency with the structure of the Leduc simulator.

Both players begin the game with \$10. At the beginning of each round, an ante (\$1) is deducted from the start of

every hand and the full pot is awarded to the winner at the end of each hand/game. For games that result in ties, the money is split between players. The simulation will terminate if either player has a balance of \$0.

E. Relevance to POMDP Training

The purpose of the simulation is to function as an environment to train a POMDP agent. Key structural properties that make it useful for training purposes include:

1. **Hidden State:** The opponents private card is unknown to the agent. The opponent's card is part of the hidden state in the POMDP formulation.
2. **Informative Observations:** Because action probabilities vary by card, the opponent's betting behavior is a noisy but informative signal about their hidden card. A POMDP agent can use this to update its belief state.
3. **Parameterized opponent:** The policy dictionary makes it easy to sweep over different opponent types during training, producing a policy that is robust rather than overfit to a single opponent style.
4. **Stochastic Transitions:** The combination of random card dealing, coin-flip acting order, and probabilistic actions means no two hands are identical, providing diverse training trajectories.

IV. SOLUTION APPROACH

A. Solution Overview

With the baseline Leduc Poker simulator now defined, we will delve into the details describing our POMDP implementation. This project implements a Partially Observable Monte Carlo Planning (POMCP) agent that learns to play Leduc Hold'Em poker against a fixed stochastic opponent over the alternative Hindsight Optimization (HOP) algorithm, due to POMCPs superior performance with Leduc Poker. The agent operates under partial observability – it never directly observes the opponent's hole card – and must infer likely opponent holdings from observed betting behavior while simultaneously planning optimal actions across both rounds of the hand. The simulation runs up to 5,000 episodes. Each episode is a fresh game starting with 10 chips per player and running until one player goes broke or 50 hands are completed. The full run produces approximately 55,000-90,000 logged hands depending on how quickly players go broke in each episode.

B. POMDP Formulation

The game is modeled as a Partially Observable Markov Decision Process (POMDP) [3] using the POMDPs.jl framework. The key design decision is that the decision point within the hand is encoded directly in the state, so a single POMCP tree search spans both rounds of the hand naturally.

The table below features the elements of the State, Observation, Action, Reward, and Transition Spaces that we hand defined for this model:

POMDP Framework			
Tuple	Representation	Variables	Description
State Space	s	P1_card, opp_card, community_card, round, last_bet, raise_count, pot, p1_stack, p2_stack, p1_contributed, opp_contributed, opp_acted_first, terminal, p1_reward	The opponent's hole card is the hidden variable. Everything else is observed by Player 1 (P1)
Observation Space	o	p1_card, community_card, opp_action, pot	P1 observes its own card, the community card (once dealt), the opponent's most recent action, and the current pot size. The opponent's hole card is never directly observed
Action Space	A	:fold, :call, :raise	Actions are context-filtered at each decision point: Fold is redirected to call (check) when no bet is on the table, since folding for free is never rational. :raise is redirected to :call when the raise cap has been reached.
Reward	R	R(s,a)	Change in P1's chip stack at the terminal state of the hand. The reward is zero for all non-terminal transitions (discount = 1.0, fully episodic within a hand).
Transition			The generative model $G(s, a) \rightarrow (s', o, r)$ simulated one full step: 1. Apply P1's action (update pot, stacks, detect fold). 2. Sample the opponent's response from the learned model. 3. If the betting round end, deal the community card (round 1 $\rightarrow$ 2) or evaluate the showdown (round 2 $\rightarrow$ terminal). 4. Return the next state, observation, and reward. The generative model is the core of POMCP – it is called thousands of times per decision point during tree search.

Table 3 POMDP Framework

*Where P1_Stack represents the amount of money held by player 1.

C. POMCP vs Hindsight Optimization

The team took two separate approaches to solve the Leduc Poker POMDP, Online Numerical Approximation (POMCP) and Formulation Approximation (Hindsight

Optimization), and evaluated which approximation performed better before implementing that algorithm with the more complex UH Leduc Poker simulation.

The first approach was Formulation Approximation, specifically utilizing HOP. The foundation of HOP, and rationale for utilizing this approximation, was the following question: “How would you play if you knew what card was in the other player’s hand?” or in other words what action would you take if you had knowledge of the future, i.e. hindsight knowledge. The following equation details how this action was selected if given an observation $o = (p1_{card}, b, p)$ where:

$b = \text{community card}, \quad p = \text{pot}, \quad \text{and} \quad \widehat{p2_{card}} = \text{sampled opponent card drawn from remaining deck:}$

$$a^* = \operatorname{argmax}_{a \in A} [\frac{1}{N} \sum R(a, p1_{card}, \widehat{p2_{card}}, b, p)] \quad (1)$$

The way in which we implemented HOP was by utilizing a fixed stochastic opponent to compete against that was the exact same as what was used for POMCP. The results of running each iteration for 55,000 hands was as follows:

Win Rate for Leduc Poker	
Hindsight Optimization	48.45%
Final 50-episode win rate	53.4%

Table 4 POMCP Vs. HOP Performance Comparison

As shown in Table 4, HOP was out performed by the POMCP. After analyzing the actions of HOP, the team noticed that the greedy nature of this approximation failed to adapt to the stochastic opponent that had the inherent ability to “bluff.” Since HOP acts as if it knows the future, or in this scenario the opponents card, it fails to account for an opponent not taking the optimal action (i.e. bluffing with a weaker hand). Yoon et al. [6] came to a similar conclusion during their work by referencing the over-optimism of anticipation of future states leading to decreased performance.

D. Particle Filter – Belief Representation

Since the opponent's card is hidden, P1 maintains a belief distribution over possible opponent holdings represented as a particle filter. The particle filter is updated throughout each hand to improve the model’s estimation of the opponent’s hidden card. The particle filter in use for this POMCP utilizes 200 particles, where each particle represents a full Leduc state with a different sampled opponent card.

After each observed opponent action, particles are reweighted by the likelihood of that action under the current opponent model. The filter is reweighted using the opening model when no prior bet is on the table, and the response model when facing a bet. Before each P1 decision, all particles are synchronized to the true current game environment (pot, stacks, round, community card) so that the tree search accounts for the given game conditions.

Additionally, a probability floor of $1e-6$ is applied to all weights to prevent zero-weight particle collapse. The particle filter resets completely at the start of each new hand, since the opponent draws a new card each hand. The prior information accumulated in the opponent model carries forward, but the per-hand belief starts fresh.

E. POMCP Planner

POMCP (Partially Observable Monte Carlo Planning) is an online planning algorithm that combines Monte Carlo Tree Search (MCTS) with particle filter belief representation. It was chosen for the following purposes:

- It handles continuous or large state spaces naturally via simulation.
- It requires only a generative model $G(s, a)$, not an explicit transition matrix, making it compatible with the complex Leduc state space.
- It naturally handles the partial observability through the particle filter representation of belief.

Below is the configuration of our POMCP Solver:

POMCP Configuration	
Solver	Basic POMCP.POMCPSolver (from BasicPOMCP.jl)
Tree Queries	$N_SIMULATIONS = 500$ per decision point
Max Depth	20 (more than sufficient for a 2-round hand)
Exploration	UCB1 with $c = \sqrt{2}$
Default Action	call (fallback if tree is empty)
RNG	MersenneTwister(42)

Table 5 POMCP Planner Configuration

Planning process:

At each P1 decision point, the planner:

1. Converts the current particle filter to a Particle Collection belief.
2. Runs 500 MCTS simulations, each starting from a particle sampled from the belief.
3. Each simulation expands the tree using UCB1 for node selection, rolls out to a terminal state via the generative model, and backpropagates the reward.
4. Returns the action with the highest estimated value at the root.

The planner is rebuilt at the start of each episode to incorporate the latest opponent model estimates into the generative model.

POMCP is not a learned policy in the traditional sense as it does not update weights between hands. Instead, it is an online planner that re-solves from scratch at each decision point. The "learning" in this system comes from the belief of the opponent model improving over time, which makes the generative model more accurate, which in turn makes the tree search explore more realistic futures.

F. Opponent Model Learning

The opponent model is a Bayesian Maximum Likelihood Estimation (MLE) that learns the stochastic opponent's policy from observed behavior over the course of the simulation.

RESULTS

The opponent model is split into two independent distributions per card, reflecting the two distinct decision contexts in Leduc:

- Opening policy — $P(\text{bet})$ when no prior bet is on the table.
 - Counts: [check_count, bet_count] per card.
 - In an opening context, fold is not a legal action, so only check and bet are meaningful to learn.
 - Response policy — $P(\text{fold/call/raise})$ when facing a bet.
 - Counts: [fold_count, call_count, raise_count] per card.
 - All three actions are available in the response context.
- This separation is critical. If opening and response observations were mixed into a single distribution, the free-fold redirections (where a fold intention is legally converted to a check) would systematically inflate fold counts and corrupt the estimates, as seen in initial model testing.
- Update rule:
 - After each hand, every opponent action entry in the action log is used to update the appropriate sub-model using MLE (add-one-style counting).
 - Crucially, the logged action is the intended action (sampled directly from the true policy distribution) before any legal correction is applied. This ensures raise-cap forced calls and free-fold redirections do not bias the learned estimates.

G. Warmup Phase

A 2,000-hand warmup phase is used to build up the learned model history before player 1 actions are decided by the POMCP algorithm. During warmup, P1 acts using a fixed policy identical to the true opponent policy rather than the POMCP planner.

If P1 used POMCP from the start, its aggressive betting style creates a systematic observational bias in the opponent model. POMCP was observed in testing to raise on nearly every hand, so the opponent almost always faces a bet rather than opening freely. The POMCP algorithm therefore was affecting its own observations of the set stochastic policy played by P2, which significantly altered the agent’s belief of the true opponent policy.

By using a set stochastic policy during warmup, P1 can accumulate unbiased observations of the opponent’s policy in many possible game states before POMCP takes control. This allows the POMCP to formulate a more realistic belief of the opponent policy before the agent’s decisions impact its observations.

As was previously discussed in this paper, the HOP algorithm was used as one attempt to solve the Leduc Poker POMDP. HOP generated a win rate of 48.65%, for comparison, the POMCP algorithm we employed produced a win rate of 63.7% when solving the Leduc Poker POMDP. Given the superior results for the POMCP algorithm, the results from that algorithm will be discussed in this section.

Card	Opponent Policy [fold, call, raise]	Learned Policy [fold, call, raise]
Jack	[(1/3), (1/3), (1/3)]	[(0.326), (0.173), (0.501)]
Queen	[(0.2), (0.5), (0.3)]	[(0.149), (0.714), (0.137)]
King	[(0.1), (0.5), (0.4)]	[(0.377), (0.352), (0.272)]

Table 6: Leduc Poker POMCP Response Policy

The table above lists the set stochastic policy that was used for the opponent throughout the simulation as well as the learned response policy produced based on the POMCP belief.

The POMCP also generated an opening policy, which was found through testing to be necessary, since not all the possible actions in the POMDP action state are truly options in the opening betting round. For instance, the player who is the first to bet would rationally never fold as it doesn’t make sense to fold for free. Likewise, if the first player had called the original ante the second player would never fold as they don’t need to wager more money in order to see the community card.

Card	Learned Opening Policy [check, bet]
Jack	[(0.84), (0.16)]
Queen	[(0.601), (0.399)]
King	[(0.625), (0.375)]

Table 7: Leduc Poker POMCP Opening Policy

As can be seen by Table 7, the opening policy the POMCP algorithm learned is a very different policy compared to the true opponent policy. This skew impacts the initial action that the POMCP chooses. The response policy is far more important to analyze as it is based on a much larger sample size and the sample size it uses has all actions in the action space available. The inaccuracies of the opening policy are why it was important to include the warmup period in the simulation for the POMCP to learn from data that wasn’t corrupted by this poorly learned policy.

Individual episode win rates are extremely noisy because episodes often end in 5-15 hands when one player goes on a run. A single lucky sequence of card deals can swing an episode from 0% to 100%-win rate regardless of policy quality. The 50-episode rolling average is a much more reliable performance indicator than any single episode result. All the win rates in this results section will be based on the 50-episode average.

H. Plot Descriptions



Figure 1: Leduc Poker POMCP Win Rate over Episodes

Top panel: During warmup, the rolling win rate is near or slightly below 50%, reflecting the set stochastic policy given to player 1. Once POMCP takes over, the rolling win rate climbs to 55–65% and remains consistently above the 50% chance baseline for the rest of the 5,000-episode run. There is no visible degradation over time, suggesting the agent does not overfit or deteriorate as the opponent model accumulates observations.

Bottom panel: The reward curve mirrors the win rate story. Average reward per hand is near zero or slightly negative during warmup (set stochastic policy), then stabilizes to a consistently positive 0.5–1.2 chips per hand once POMCP takes over. The occasional spikes correspond to episodes where P1 went on a winning streak and bankrupted the opponent quickly.

These two plots clearly show that POMCP easily outperforms the initial set policy. The transition from warmup to POMCP control is clearly visible as a step-change in performance. After the initial step up, performance is stable across the remaining episodes.

I. UH Leduc Poker Results

After the success of the POMCP algorithm for the simple 6-card Leduc poker, the team decided to update the policy for 18-card UH-Leduc poker. This involved an updated opponent policy, POMDP state space, and an updated particle filter to handle the increased state space. With this larger POMDP problem to handle, the assumption was that the POMCP algorithm wouldn't perform as dominantly as before. The 50-episode win rate seemed to only slightly confirm that assumption as the UH-Leduc 50-episode average win rate regularly reached 58%. As was done before with the Leduc poker simulator, a separate opening and response policy was set by the POMCP algorithm.

Card	Learned Policy [check, bet]
Jack	[(0.497), (0.503)]
Queen	[(0.402), (0.598)]

King	[(0.505), (0.495)]
Ace	[(0.252), (0.748)]

Table 8: UH Leduc Poker Opening Policy

As was seen previously in the Leduc poker simulator results, the UH-Leduc poker results for the opening policy learned by the POMCP algorithm before all actions are available is far from the true opponent policy. This was corrected again by utilizing a warmup period to minimize the impact of the opening policy results on the far more important response policy, shown below.

Card	Opponent Policy	Learned Policy [fold, call, raise]
Jack	[(1/3), (1/3), (1/3)]	[(0.328), (0.344), (0.329)]
Queen	[(0.2), (0.5), (0.3)]	[(0.202), (0.501), (0.298)]
King	[(0.1), (0.5), (0.4)]	[(0.099), (0.494), (0.406)]
Aces	[(0.06), (0.42), (0.52)]	[(0.061), (0.421), (0.518)]

Table 9: UH Leduc Poker Response Policy

In the table above, the stochastic opponent policy used throughout the simulation is compared against the respective learned response policy based on the UH-Leduc Poker POMCP belief. In contrast to the Leduc Poker learned policy, the UH-Leduc Poker learned policy was almost able to reach the identical state of the stochastic opponent policy implemented. So the POMCP algorithm was able to reach a more accurate belief of the opponent policy for the UH Leduc Poker POMDP, however, the algorithm had a slightly worse win rate as compared to the win rate for simplified Leduc Poker. This contradiction in results shows the difficulty of learning Poker given the strategy involved that may not be easy to learn or predict.

J. UH Leduc Poker Plot Descriptions

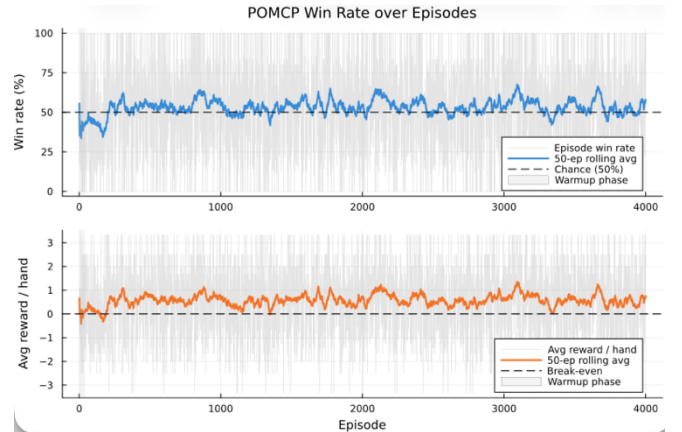


Table 10: UH Leduc POMCP Win Rate over Episodes

Top panel: Compared to the Leduc Poker model, the warmup rolling win rate is below 50% by a more significant margin, reflecting the set stochastic policy given to player 1. Once POMCP takes over, the rolling win rate settles around 50–65%, a noticeably larger variance as compared to Leduc Poker, and hugs the 50% chance baseline for the rest of the

5,000-episode run with outliers jumping to the upper bound. There is no visible degradation over time, suggesting the agent does not overfit or deteriorate as the opponent model accumulates observations.

Bottom panel: The reward curve mirrors the win rate story by performing better than a net draw, average return of zero, but not reaching the heights of the Leduc Poker mode. Average reward per hand is near zero or slightly negative during warmup (set stochastic policy), then stabilizes to a consistently positive 0.3-1.0 chips per hand once POMCP takes over. The occasional spikes correspond to episodes where P1 went on a winning streak and bankrupted the opponent quickly.

Even though the POMCP struggled more with UH Leduc Poker, which was expected due to the increased complexity, the two plots clearly show that POMCP still outperforms the initial set policy. The transition from warmup to POMCP reinforces the visible step-change in performance. After the initial step up, performance is stable but with a larger variance across the remaining episodes.

V. CONCLUSION

Poker, and its variants, provide a unique POMDP formulation environment to test different POMDP solution algorithms. After our implementation of the POMCP algorithm for UH Leduc Poker, we are satisfied with the algorithm's performance, as it consistently achieved over a 50% win-rate, against a set stochastic opponent policy. Given these results, we believe that POMCP is a successful solution for POMDPs formulated from the game-state of poker.

One thing to note from our testing, is it became apparent that due to the randomness included in our simulations, as well as in Monte Carlo Planning as an algorithm, the results we detail in this paper are not repeatable on every given run. Luck of the draw, true to the game of poker, is a factor in our results, and some runs may produce higher or lower average win rates as compared to what we have reported here.

If we had more time to work on this project, our next step would be to increase the scale of our state space from an 18-card deck to a full 52-card deck. This would also involve expanding the policy definitions and increasing the size of the POMCP particle filter to account for the increased state space. It would be interesting to see how the POMCP would perform with its simplified game structure, but the full-sized deck considering how similar the opponent's policy would have to be for many of the possible card options. The next step would be to attempt to expand the simulation to allow for more players, more community cards, and more betting rounds. These would all increase the complexity of the simulation and would add more noise for the agent to parse through to find a winning policy.

If work were continued on this project to enhance the current model, we have considered several additional approaches that could improve the simulation:

1. The first would include creating an opponent policy that is not as restrictive as the current one.
2. Separate Warmup Episodes:
Rather than mixing warmup and POMCP hands within the same episode counter, a dedicated pre-training phase with a fixed number of warmup episodes would allow cleaner separation of the learning and exploitation phases.
3. Adaptive POMCP Parameters:
Increasing $N_{\text{SIMULATIONS}}$ over time as the opponent model converges could improve late-game decision quality. Early in training, tree search quality is limited by model accuracy, not simulation count.
4. Opponent Adaptation:
The current opponent model assumes a stationary opponent policy. Extending it to detect and adapt to policy changes would make the agent more robust in settings where the opponent learns over time.

Lastly, we would update the model to accommodate the full 52 card game of Texas Hold'em, demonstrating that the simulator can accommodate a complete game with multiple agents.

VI. CONTRIBUTION/OFF-THE-SHELF USE

This project was completed by Mark Jaschke, Cristian Montelongo, and Ikenna Odelewe. Mark's Contribution to the project included the creation of the Leduc Poker and POMCP Simulators. Cristian developed the alternative Hindsight Optimization simulator to be used as a comparison model to the POMCP. Ikenna gathered the information compiled from the developed models and produced this publication. The authors all agree to allowing this paper to be posted for the benefit of future DMU students.

We did NOT use any off-the-shelf libraries as we created a starting simulation/solver of Leduc Poker and then iterated from there to attempt to handle the University of Houston [3] version of Leduc Poker.

REFERENCES

- [1] M. Bowling, N. Burch, M. Johanson, and O. Tammelin, "Heads-Up Limit Hold'em Poker is Solved," *American Association of Advancement of Science*, pp. 145-149, Jan. 2015.
- [2] A. Gilpin, and T. Sandholm, "Optimal Rhode Island Hold'em Poker," thesis, 2005.
- [3] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray, *Algorithms for Decision Making*. Cambridge, MA: The MIT Press.
- [4] S. R. Parth, "A Reinforcement Learning Approach For UH Leduc Poker," thesis, 2021.

- [5] D. Silver and J. Veness, "Monte-Carlo Planning in Large POMDPs," *Advances in Neural Information Processing Systems*, vol. 23.
- [6] S. Yoo, A. Fern, R. Givan, and S. Kambhampati, "Probabilistic Planning via Determinization in Hindsight," *AAAI Conference on Artificial Intelligence*, no. 23, pp. 1010-1016, 2008.